OPERATIONS GUIDE

Technical Baseline Scorecard

A controlled starting-point assessment for coding, system design, explanation quality, and the first correction to test.

Measure observable interview behavior under consistent conditions without turning one attempt into a readiness verdict.

VERSION 1.0

FORMAT GUIDE + WORKSHEET

STATUS LEARNER READY

OPERATIONS / TECHNICAL-BASELINE-SCORECARD

MEASURE THE START

A baseline captures behavior, not identity

Use the same conditions, observable dimensions, and evidence notes so the next attempt can show whether a specific correction worked.

CORE IDEA

The baseline is a starting measurement, not a readiness score. Record what happened, where the explanation broke, and the earliest decision that would improve the next attempt.

01
Prompt
One coding prompt and one system-design prompt.

02
Conditions
Fixed time, tools, recording method, and no hidden assistance.

03
Evidence
Quotes, timestamps, tests, assumptions, and missed decisions.

04
Correction
One behavior to front-load in the next comparable attempt.

DO
Score the behavior
Clarification, decision order, invariant or requirements, correctness, trade-offs, tests, failure handling, and explanation.

DO NOT
Avoid the label
Do not turn one attempt into 'good at coding,' 'bad at design,' or a fabricated percent-ready number.

Three-point evidence scale

Score	Meaning	Evidence standard
0 - Missing	The behavior did not appear when it was needed.	Name the moment and consequence.
1 - Inconsistent	It appeared late, needed prompting, or was incomplete.	Quote or timestamp the partial behavior.
2 - Reliable	It appeared unaided, at the right moment, and changed the answer.	Record the decision and resulting quality.

CODING BASELINE

Score the decisions that make a solution trustworthy

Use the neutral prompt below so the attempt is reproducible. The goal is to observe the full interview sequence, not maximize difficulty.

BASELINE PROMPT

Given a string, return the length of its longest contiguous substring with no repeated characters. Explain your state and invariant, then test empty input, all-identical characters, and a repeat that occurs after a gap.

Dimension	What reliable behavior sounds or looks like	0 / 1 / 2
Clarification	Restates inputs, outputs, constraints, and examples before selecting an approach.	
Pattern choice	Connects a prompt cue to an approach and rejects one plausible alternative.	
State / invariant	Names what is stored and what remains true after each step.	
Correctness	Implements the stated approach without changing the contract silently.	
Complexity	Explains time and space from the operations and state actually used.	
Testing	Covers empty/minimum, boundary, duplicate, and main-transition cases.	
Communication	Keeps decisions audible without narrating every keystroke.	

Prompt setup

30-MINUTE RUN

Recommended timing

5 minutes clarify and choose; 20 minutes implement; 5 minutes test and explain complexity.

RECORD

Evidence to save

Prompt, approach, invariant, code or notes, first missed edge, and one exact quote where reasoning became unclear.

SYSTEM-DESIGN BASELINE

Score the decisions that keep architecture explainable

Use the neutral prompt below so the baseline tests reasoning order and trade-offs rather than trivia about a particular technology.

BASELINE PROMPT

Design a notification-delivery service for product events. Support in-app and email delivery, user preferences, retries without duplicate visible sends, and delivery status. Assume 10 million monthly active users and a peak of 5,000 events per second; clarify any missing product promise before designing.

Dimension	What reliable behavior sounds or looks like	0 / 1 / 2
Requirements	Separates critical actions, quality attributes, and explicit non-goals.	
Scale model	Uses order-of-magnitude traffic, payload, peak, and retention assumptions.	
Decision order	Moves from contract and data flow to components; boxes have stated reasons.	
Data and state	Names source of truth, entities, invariants, partitioning, and consistency.	
Trade-offs	Compares a credible alternative and ties the choice to a requirement.	
Failure behavior	Explains timeout, retry, idempotency, overload, and degraded operation.	
Operations	Closes with the first bottleneck, metric, alert, and recovery test.	

Prompt setup

35-MINUTE RUN

Recommended timing

5 minutes requirements; 5 scale and API; 15 architecture/data; 10 failure, trade-offs, and close.

RECORD

Evidence to save

Requirements, estimates, architecture sketch, three assumptions, first bottleneck, one rejected alternative, and the first unclear transition.

WORKED EXAMPLE

Turn an observation into one testable correction

This fictional example shows the difference between a vague weakness label and evidence that can guide the next attempt.

Observation	Weak interpretation	Evidence-led interpretation	Next correction
Chose a hash map quickly but never stated what the values represented.	I am weak at hash maps.	Pattern choice was timely; the state invariant stayed implicit, so duplicate handling became confused.	Before coding, say: map stores value -> last index; query before insert.
Drew a queue before estimating peak load.	My architecture is shallow.	The component appeared before the load/failure requirement that justified it, making capacity and retry policy hard to defend.	State peak arrival, service rate, and retry bound before drawing the queue.
Explained the answer after finishing rather than during decisions.	I communicate badly.	Decision moments were silent; the final summary was clear but too late for an interviewer to follow the reasoning.	Use a one-sentence decision checkpoint before each major move.

CORRECTION RULE

Choose the earliest missing behavior that caused later confusion. A small front-loaded correction is easier to test than a broad promise to 'study more.'

A correction is testable when

- ☐ It can be observed at a specific moment in the next attempt.
- ☐ The prompt and rubric stay comparable.
- ☐ Success does not depend on knowing the exact same answer in advance.
- ☐ You can name what stronger evidence would look like.

WORKSHEET

Record two baseline attempts

Complete this page immediately after each attempt. Use short quotes and timestamps instead of reconstructing the experience later.

Setup	Coding attempt	System-design attempt
Prompt		
Time / tools		
Strongest observable behavior		
First unclear or missing decision		
Quote / timestamp / artifact		
Consequence in the answer		

Selected first correction (one total)	Where it must appear	Evidence of success

FINISH THE TASK

Choose one correction and schedule the re-test

The baseline is complete when the next attempt has a defined behavior to test - not when every dimension receives a high score.

01	02	03	04
Run	**Observe**	**Select**	**Re-test**
Use fixed conditions and save the artifacts.	Score dimensions and quote the first breakdown.	Choose one early behavior with downstream impact.	Use a comparable prompt within 48-72 hours.

Do not compare totals blindly

A coding 10 and a design 10 do not mean the same thing, and a two-point movement can be noise if prompt difficulty or conditions changed. Compare dimension evidence and decision moments, not a single combined number.

Common mistakes

+ Choosing a prompt so hard that no complete interview sequence appears.
+ Changing the rubric after seeing an uncomfortable result.
+ Scoring from memory without quotes, timestamps, tests, or artifacts.
+ Selecting several corrections, which makes improvement impossible to attribute.
+ Treating a baseline as a prediction of hiring outcome.

YOUR NEXT ACTION

Write the correction as an observable sentence, choose the comparable re-test prompt now, and place it on the calendar before returning to the roadmap.

Completion check

- [ ] Two prompts were attempted under recorded conditions.
- [ ] Both scorecards contain evidence, not only numbers.
- [ ] The earliest important breakdown is named.
- [ ] One correction and one re-test condition are scheduled.